# Does Hardware Change Which Generative Model Is Efficient?

Training-Free Acceleration of Diffusion and Flow-Matching Transformers Across GPU Generations

Iason-Daniil Zakynthinos

***Abstract*—Sampling efficiency of image generators is usually reported as the number of function evaluations (NFE) or as a timing on one accelerator. NFE is a sound proxy only while every network call costs the same, but the training-free accelerations now used in practice, namely feature caching and low-precision kernels, change the cost of a call while leaving NFE unchanged, and their benefit depends on what the hardware supports. This project measures whether quality–cost rankings of diffusion and flow-matching samplers, with and without such accelerations, transfer across three GPU generations. For each device and batch size I will build Pareto frontiers from Fréchet Inception Distance (FID), precision/recall, latency, and energy per image, and quantify how far NFE and FLOPs mispredict the measured ranking.**

## I. Research Question and Hypotheses

*When an acceleration changes the cost of a network call but not the number of calls, do the resulting quality–cost rankings of diffusion and flow-matching samplers survive a change of GPU?*

**H1 (control).** Without acceleration, latency is linear in NFE on every device and rankings are stable. DiT and SiT share the same architecture and GFLOPs [1], and solver overhead is small next to a transformer forward pass, so this is expected; it confirms the harness and isolates the effect of the accelerations.

**H2 (rankings move).** Speedups from feature caching and from INT8/FP8 kernels depend on the device and batch size: low-precision kernels help most when the GPU is compute-bound and not at all where they are unsupported, while caching helps less when a small batch leaves the GPU launch-bound. Adding these configurations should therefore lower cross-device rank agreement and change Pareto-frontier membership, which neither NFE nor FLOPs can predict.

**H3 (who tolerates caching).** Caching reuses features computed at a neighbouring step, so it should hurt least where consecutive network outputs are most similar. I expect the flow-matching model, whose learned paths are designed to be straighter [2], to lose less quality under caching than the diffusion model, and both to lose more at 8 steps than at 32. At 8 steps there is little redundancy between steps to exploit, so I predict that caching fails there, in quality, in speed, or both; this is a stated prediction to test, not a risk to the project.

## II. Motivation and Related Work

Diffusion models generate images by iterative denoising [3]; flow matching learns a velocity field along simpler probability paths [2]. DiT [4] and SiT [1] form a controlled pair: one transformer backbone, two training objectives. Fast solvers cut sampling to tens of evaluations [5], [6], and newer training-free methods lower the cost of each evaluation instead: feature caching reuses intermediate activations across steps [7], [8], and post-training quantization runs DiTs at 8 bits with little quality loss [9]. These methods are evaluated on one GPU and reported as NFE, FLOPs, or a single speedup factor, and the factors are fragile: FORA gave no speedup ($0.99\times$) in a recent batch-1 comparison on an RTX 5090 [10], implementations of the same caching method differ in latency on one GPU [11], and a recent survey names cross-platform behaviour as an open problem [12]. FLOPs and parameter counts are known to disagree with measured efficiency [13], and inference-energy benchmarks show large differences between models and GPUs [14], but they do not ask whether a diffusion-versus-flow or accelerated-versus-plain conclusion drawn on one GPU still holds on another.

## III. Methodology

**Models and factors.** Public ImageNet-256 checkpoints of DiT-XL/2 and SiT-XL/2; nothing is trained. Samplers: DDIM and DPM-Solver for DiT, Euler and Heun ODE for SiT. Steps: 8, 16, 32. Acceleration: none (fp16), FORA-style feature caching with a fixed reuse interval [8], INT8 W8A8, and FP8 W8A8. This gives $2 \times 2 \times 3 \times 4 = 48$ configurations. Batch sizes 1 and 16 are a timing factor. Classifier-free guidance scale, compile settings, seeds, and class labels are fixed; every network call counts toward NFE, and FLOPs per image are counted separately, so cached calls are visible as cheaper calls.

**Hardware.** Three generations, one GPU per run: an NVIDIA RTX 5080 (Blackwell, local, the main machine), a Google Colab T4 (Turing, no bf16 or FP8), and one GH200 on the Ikaros cluster (Hopper, limited hours). A kernel smoke test in week 1 runs fp16, INT8, and FP8 on each GPU; where a device lacks a kernel, as with FP8 on the T4, that configuration is fixed as unavailable before the sweep, and the gap is part of the result.

**Protocol.** Kernel warm-up, repeated CUDA-event timings reported as median and interquartile range, peak memory, and energy from the NVML counter, checking on the GH200 that it reports GPU rather than whole-module power.

**Software stack.** The stack cannot be fully identical across devices: Colab fixes its own environment, the RTX 5080 needs a recent CUDA release, and the GH200 host is ARM. To keep hardware the only intended variable, every method uses one in-house implementation, the same PyTorch and torchao versions

are pinned wherever they are available, and exact versions are logged with every run. Any remaining difference is reported as a threat to validity.

**Quality.** All images are generated on the RTX 5080, which supports every configuration. Quality is a property of the configuration, not the device, and a subset is regenerated on the other two GPUs to confirm that kernel differences do not change it. A pilot of 2,000 images per configuration uses clean-fid with fixed ImageNet statistics [15] and improved precision/recall [16]. Configurations on or near the pilot frontier are re-run with 10,000 images and three seeds. FID is compared only at equal sample counts, because it is biased at small $N$ [17].

## IV. Analysis, Feasibility, and Expected Contribution

**H1/H2.** Per device and batch size, regress latency and energy on NFE and on FLOPs, and report the error by acceleration type. Build FID–latency and FID–energy frontiers, compute Kendall's $\tau$ between devices for plain and accelerated configurations with bootstrap intervals, and list every configuration that is on the frontier on one device but dominated on another. **H3.** Compare the FID increase caused by caching across family and step count. As a mechanism check, relate it to the cosine similarity of consecutive-step block outputs, which is cheap to log during sampling.

**Feasibility.** About 6 GPU-hours on the RTX 5080 cover the 48-configuration pilot; the confirmation stage adds about a day. Cross-device work consists of short timing sweeps. If compute is short, cut in order: FP8, then batch 16 on Ikaros, then the 32-step level for accelerated configurations. Both families, caching, and the cross-device timing are never cut.

**Deliverables.** (i) An open benchmark harness and configuration manifest, (ii) per-device Pareto frontiers and rank-agreement statistics, and (iii) evidence on when NFE or FLOPs can stand in for measured cost and when an efficiency claim must state its hardware. Either outcome is informative. Serving systems, training, and exhaustive solver searches are out of scope.

## V. Proposed Schedule

*Weeks 1–2:* checkpoints, pinned environment on all three GPUs, protocol, and the kernel smoke test. *3–4:* timing, energy, and FLOP counting on the RTX 5080; caching and INT8/FP8 paths validated against fp16 outputs. *5–7:* 2,000-image pilot, FID and precision/recall, preliminary frontiers. *8–9:* timing sweeps on T4 and GH200 and cross-device output checks. *10–11:* 10,000-image, three-seed confirmation. *12–13:* analysis, plots, report.

## References


[1] N. Ma, M. Goldstein, M. S. Albergo, N. M. Boffi, E. Vanden-Eijnden, and S. Xie, "SiT: Exploring flow and diffusion-based generative models with scalable interpolant transformers," in *Proc. ECCV*, 2024.

[2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, "Flow matching for generative modeling," in *Proc. ICLR*, 2023.

[3] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," in *Proc. NeurIPS*, 2020.

[4] W. Peebles and S. Xie, "Scalable diffusion models with transformers," in *Proc. ICCV*, 2023.

[5] J. Song, C. Meng, and S. Ermon, "Denoising diffusion implicit models," in *Proc. ICLR*, 2021.

[6] C. Lu, Y. Zhou, F. Bao, J. Chen, C. Li, and J. Zhu, "DPM-Solver: A fast ODE solver for diffusion probabilistic model sampling in around 10 steps," in *Proc. NeurIPS*, 2022.

[7] X. Ma, G. Fang, and X. Wang, "DeepCache: Accelerating diffusion models for free," in *Proc. CVPR*, 2024.

[8] P. Selvaraju, T. Ding, T. Chen, I. Zharkov, and L. Liang, "FORA: Fast-forward caching in diffusion transformer acceleration," arXiv:2407.01425, 2024.

[9] J. Wu, H. Wang, Y. Shang, M. Shah, and Y. Yan, "PTQ4DiT: Post-training quantization for diffusion transformers," in *Proc. NeurIPS*, 2024.

[10] X. Yin, C. Xu, H. Zhou, B. Wei, and W. Cai, "AccelAes: Accelerating diffusion transformers for training-free aesthetic-enhanced image generation," in *Proc. ECCV*, 2026.

[11] A. Aggarwal, A. Shrivastava, and M. Gwilliam, "Evolutionary caching to accelerate your off-the-shelf diffusion model," in *Proc. ICLR*, 2026.

[12] J. Liu *et al.*, "A survey on cache methods in diffusion models: Toward efficient multi-modal generation," arXiv:2510.19755, 2025.

[13] M. Dehghani, A. Arnab, L. Beyer, A. Vaswani, and Y. Tay, "The efficiency misnomer," in *Proc. ICLR*, 2022.

[14] J.-W. Chung, J. Liu, J. J. Ma, R. Wu, O. J. Kweon, Y. Xia, Z. Wu, and M. Chowdhury, "The ML.ENERGY benchmark: Toward automated inference energy measurement and optimization," in *Proc. NeurIPS Datasets and Benchmarks Track*, 2025.

[15] G. Parmar, R. Zhang, and J.-Y. Zhu, "On aliased resizing and surprising subtleties in GAN evaluation," in *Proc. CVPR*, 2022.

[16] T. Kynkäänniemi, T. Karras, S. Laine, J. Lehtinen, and T. Aila, "Improved precision and recall metric for assessing generative models," in *Proc. NeurIPS*, 2019.

[17] M. J. Chong and D. Forsyth, "Effectively unbiased FID and Inception Score and where to find them," in *Proc. CVPR*, 2020.